MMA3001 Assignment - Predicting Building Occupancy

Sebastian Esposito
Repository: https://github.com/SebastianE100/MMA3001-Project.git

October 2026

1 Engineering problem

The goal of this project is to use building occupancy data to build a model that predicts the number of people present in the building to minimise energy usage of heating and cooling systems. To reduce the amount of energy used by heating and cooling, it is crucial to only operate these systems when they are needed. As a result, the model should put particular emphasis on accurately predicting when the building is empty so the automated systems can be turned off. A secondary goal of the model should be to accurately predict the occupancy of the building at any time to ensure the building is at sufficient temperature when people arrive. The data used for this project was supplied from Monash Smart Infrastructure Occupancy and Environmental Data.

A limitation of the model used is that the building was specified to be vacant if two people or less were present. This definition is a requirement of this modelling approach due to the limitation of a model to predict exactly zero for a given input. The model is fitted to one building and data over a period from November 2023 to April 2026, so its accuracy for other buildings or semesters is untested. The model only has limited context and cannot anticipate public holidays, events or weather. Heating and cooling systems are unable to quickly change the temperature, thus the model should only predict the maximum number of people present in a one hour interval.

2 Inputs and outputs

Data preparation is separated from model training and then used in multiple scripts to create and validate the model. The raw data is stored in a CSV file of about 9 million rows that must be opened, and data such as timestamps and headcounts are extracted. To reduce the computational load every time the data is required, the data extraction is done as a preprocessing step. The raw data is collated and filtered to be outputted as a Parquet file. That file is then opened by the modelling scripts that produce and validate the model by producing predicted headcounts and performance plots.

2.1 Preprocessing

To decrease runtime, the DataPrep script only reads the columns collecteddate and headcount from the raw data CSV file. The data undergoes filtering, which removes rows with duplicate timestamps, extracts the day of the week the sample was taken and resamples the data to be the maximum recorded in 1-hour intervals. The script sets intervals without a reading to be zero under the assumption that the lack of raw overnight values is due to sensors stopping to record. However, the resample data set is filtered to remove days where no readings were recorded. A limitation of this approach is that if a sensor is disabled throughout the day, those times will remain as zeros. The script then assigns Class and Exam flags to each interval using teaching and exam period dates from the Monash University academic calendar to provide calender context to models. The result is written to PeopleData.parquet for use by downstream scripts.

Inputs	Description	Units
headcount	CSV File containing column that stores occupancy of the building	Integer
collecteddate	CSV File containing column that stores date and time of measurements	Datetime
Semester Dates	University semester dates are inputted to creates contextual flags	Datetime

Table 1: Pre-processing Script Input

Outputs	Description	Units
collecteddate	Dateframe column that stores date and time of measurements	datetime
headcount	Dateframe column that stores occupancy of the building	Integer
Day	Dateframe column that stores the day of the week	String
Class	Dateframe column that stores flag if classes are running	Boolean
Exam	Dateframe column that stores flag if exams are running	Boolean
Time	Dateframe column that stores the time of day in hours	Float

Table 2: Pre-processing Script Output

2.2 Modelling Scripts

There are multiple modelling scripts that use the previously mentioned Parquet file to perform analysis to validate model performance. A summary of the different files can be found in the Table **??**.

Script	Description
Base Line.py	Evaluates baseline models
Complexity Investigation.py	Evaluates inputs for model
Depth Sweep.py	Evaluates optimal depth for model
Regression Model.py	Evaluates alternative regression model

Table 3: Modelling scripts used in this project

The above files take PeopleData.parquet as an input to output the plots and data seen later in this report.

2.3 Final Model

After evaluating many different solutions, the final model that was selected accepts inputs of the day and the time of the day to predict the headcount of an inputted hour. This model is trained on the day and time data received from PeopleData.parquet using a chronological time series split and a weight of 65 for values when the building is empty.

Model Inputs	Description	Units
dayNum	The day of the week as a number (Monday=0 through to Sunday=6)	integer
time	The number of hours that has passed in the day	Float

Table 4: Final Model Inputs

Model Outputs	Description	Units
Prediction	Returns the expected occupancy of the building	Float

Table 5: Final Model Output

3 Computational solution

The final selected model was to use a weighted decision tree with inputs of day of the week and time of day. A decision tree works by repeatedly splitting the data into groups so that the error within each group is minimised. This makes it well suited to this application, because it can adapt to sharp changes in the data, such as when a class begins and occupancy increases rapidly. To make a more informed prediction, providing the model with added context about the time of year was investigated. To provide this, class and exam flags are passed to the model, allowing it to differentiate between busy and quiet periods in the building. However, as seen in the Optimisation and Performance section, this added complexity did not improve performance. Additionally, this model was further optimised by performing a depth and weight sweep.

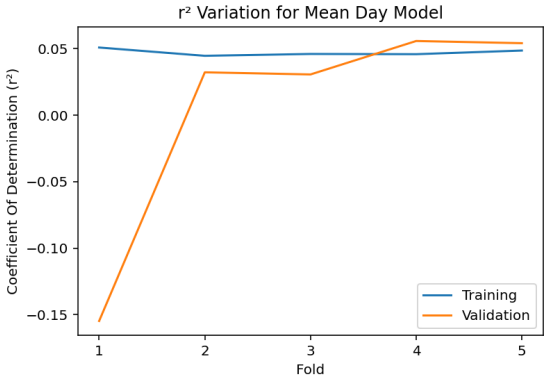
The settings and parameters of this decision tree model that were tested include the inputs, depth, minimum leaf size and training weights. Different inputs were given to the model, such as the day of the week, time of day, and context about the university schedule. One of the most important parameters of a decision tree model is the allowable depth of the tree. An increase in depth allows the model to more closely fit the data and increases the number of unique values the model can output. For this study, the minimum leaf size did not have significant influence on the model, due to the model always being depth limited due to the large amount of data points. Additionally, the weight during model fitting was changed to prioritise predicting periods when the building was empty. For further information about how these parameters were implemented, please see the Optimisation and Performance section.

The large variance in the building occupancy data has greatly limited the model that has been developed. The data is expected to have a large variance due to the many factors that influence the occupancy of a building for any given time, such as time of year, events and weather. As a result, it is challenging for a model to determine the underlying trend without knowledge of those extra factors. To combat this, it was attempted to give context to the model through the use of class and exam flags that are raised for times that are in the teaching or exam period. There is scope for further improvement by training a new model with building specific factors allowing a model to predict occupancy based on knowledge of future events rather than mean behaviour of days and times.

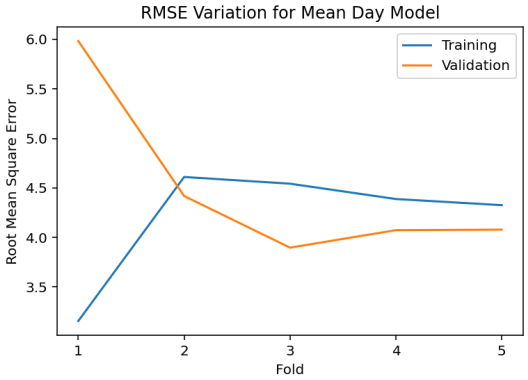
4 Alternative solutions

4.1 Baseline Model

To develop a model, a simple baseline model must be selected so all potential models have a fixed benchmark to compare to. Additionally, a baseline model allows more complex models to be rejected if it cannot improve on the simpler model. For this project, it was investigated if a model based on the mean value of each day of the week or based on the mean value for the time of day would be a more appropriate baseline model. Both these solutions are computationally inexpensive, as they only store a fixed amount of mean values. The potential baseline model's performance was assessed by using chronological time-oriented splits to form training and validation data.

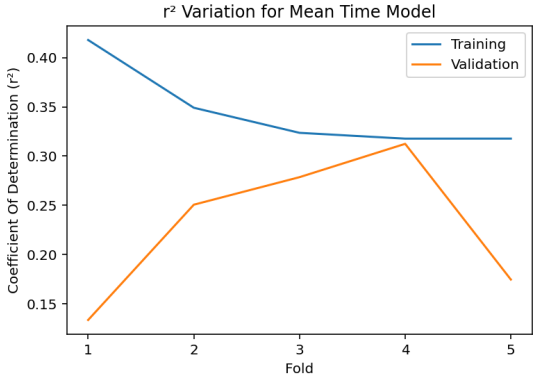


(a) Coefficient of Determination

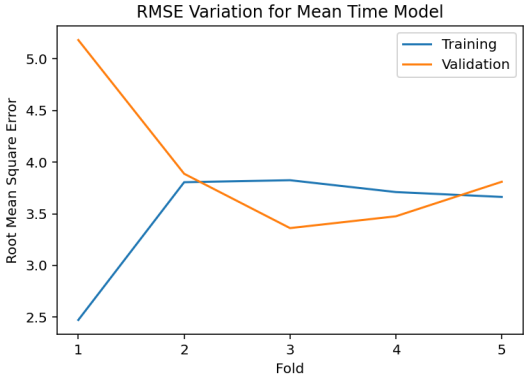


(b) Root Mean Square Error

Figure 1: Performance of Mean Day Model



(a) Coefficient of Determination



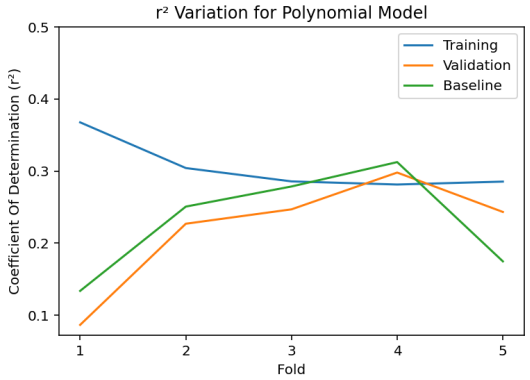
(b) Root Mean Square Error

Figure 2: Performance of Meant Time Model

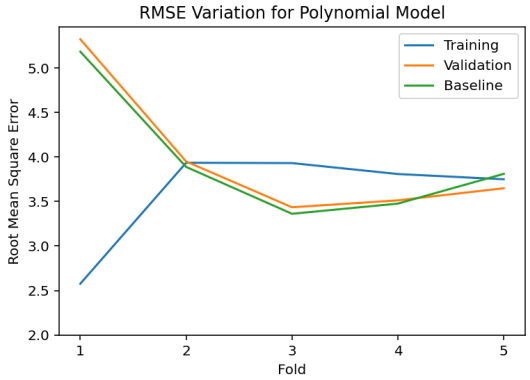
By comparing Figure 1 and Figure 2 it can be seen that the Mean Time model better captures the variance of the data set, while the Mean Day model fails to account for any significant variance. The mean time model also exhibits a lower RMSE error compared to the Mean Day model. However, it was not expected that the variation in RMSE has approximately the same shape for both models. Moreover, the validation error should not be less than the training error for folds 3 and 4. This unusual behaviour is investigated in the Validation section of the report. Overall, the Mean Time model was selected to be the baseline for this project due to its improved performance at capturing variance and reducing error.

4.2 Polynomial Regression Model

The alternative solution of a polynomial regression model in place of the decision tree was considered in this project. A polynomial model is desirable because it is a simple and efficient method to handle large amounts of data. However, it is limited in its flexibility to represent non-linear behaviour and directly use categorical variables. This project seeks to overcome these limitations by fitting a degree two polynomial to each categorical variable of the day of the week. The global model made predictions by selecting the polynomial that corresponded with the day that was being predicted. A degree three polynomial was briefly explored in this investigation, however it added significant complexity and increased the runtime of the model. However, this runtime increase is likely caused by the implementation of the model and could be improved in the future. Additionally, future development can explore other functions that can better capture the periodic non-linear relationships present in the dataset.

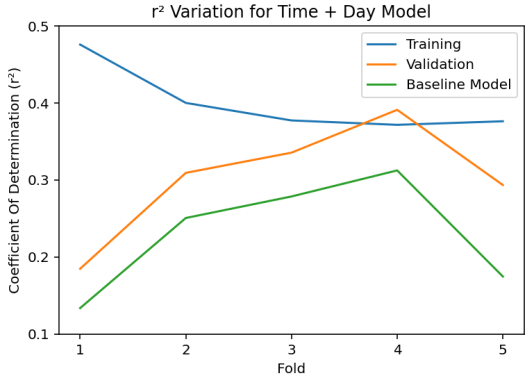


(a) Coefficient of Determination

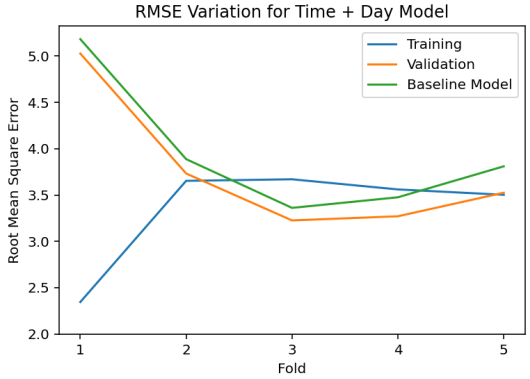


(b) Root Mean Square Error

Figure 3: Performance of Polynomial Model



(a) Coefficient of Determination



(b) Root Mean Square Error

Figure 4: Performance of Decision Tree Model

Figure 8a and Figure 4 show the polynomial model performs approximately equally to the baseline model. This is expected due to the large variation in the dataset, meaning the model regresses to the average value for each hour, which is the same process used to create the baseline model. While the performance of this model could be improved by different regression models, this approach was not pursued due to issues with numerical stability and scalability.

Metric	Decision Tree	Polynomial Model
Fitting Time (s)	Mean:1.1 Std: 0.07	Mean: 1.6 Std: 0.08
Prediction Time (s)	Mean: 0.002 Std: 9e-05	Mean: 0.18 Std: 0.01
Memory	8 branches of depth 3	7 quadratic polynomials with 3 coefficients each
Coverage	20785 Data Points	20785 Data Points

Table 6: Comparison of Polynomial and Decision Tree Approaches

To improve both the decision tree and polynomial model, more context of the time of the year could be provided. Long periods of low and high occupancy, such as the university teaching period and holidays, vary every year, making them challenging for models to predict. This information could be provided to the model to improve performance, as it is known well in advanced. With an increase in the amount of information available, the polynomial model will need to perform more fits for each combination of categories, dramatically increasing training time and run time. Additionally, if a combination of categories is not present

in the training data, the polynomial model is unable to make any prediction if the combination appears in the future. Additionally, as seen in Table 6, a decision tree method is already approximately about 90 times faster to predict and about 1.5 times faster to fit than the regression model. Overall, a decision tree model was selected due to its increased efficiency and greater ability to be scaled and improved.

5 Validation

A successful model must balance its ability to achieve the primary goal of predicting periods when the building is unoccupied and the secondary goal of an accurate prediction of the number of occupants. For this project, if a model could predict that the building is empty 75% of the time when it is actually empty, it was considered to be successful. Additionally, a model is considered to achieve the secondary goal if it can capture 50% of the variance of the occupancy of the building, despite the high variance present in the data. Ultimately the final model achieved the primary goal, as seen in Figure 9 after a weight of 65 was selected. However, the model never achieved the secondary goal, meaning that the current model is suited to measure if the building is occupied and not accurately predict the occupancy.

As buildings can have extended periods of high and low occupancy, it is crucial to account for how this variation impacts model performance. Initially, this project used a time-ordered 80-20 split to create the training and validation data to not leak future data to the model during training. However, it was observed that validation error was consistently lower than the training error. As a result, the project switched to using chronological cross-validation to evaluate model performance. This was done to assess model performance separately from the training data used. Despite this, there were folds where validation error was lower than training error, as seen in folds three and four in Figures 1-4.

Fold	Mean Occupancy	Standard Deviation
Fold 1 Train	3.48	3.24
Fold 1 Val	5.94	5.57
Fold 2 Train	4.52	4.72
Fold 2 Val	4.64	4.49
Fold 3 Train	4.58	4.65
Fold 3 Val	4.17	4.19
Fold 4 Train	4.45	4.49
Fold 4 Val	4.11	4.19
Fold 5 Train	4.35	4.19
Fold 5 Val	4.44	4.35

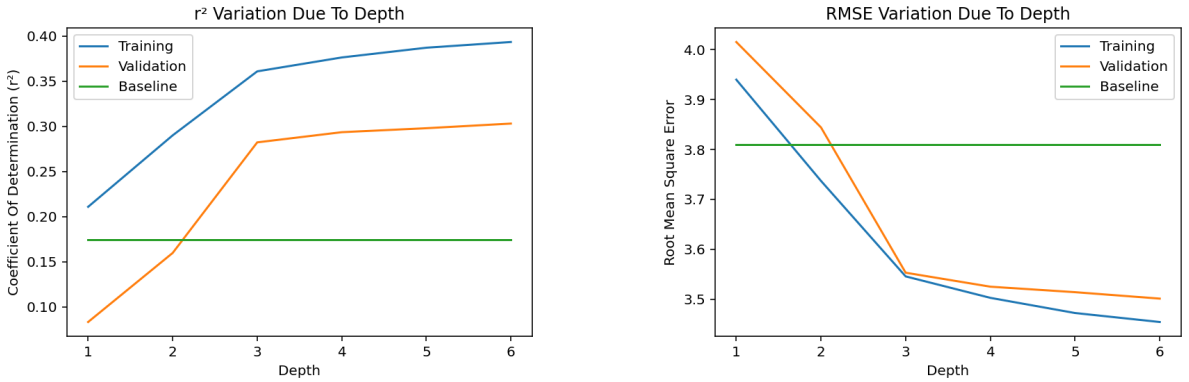
Table 7: Investigation in fold distribution

As shown in Table 7, the mean headcount in the training data is 10% and 8% higher than in the validation data for folds 3 and 4 respectively. These higher training means are likely caused by short, sharp increases in occupancy that the decision tree lacks the flexibility to capture. Because root mean square error penalises large errors heavily, these unexplained peaks inflate the training error relative to the validation error. The training and validation data also differ markedly in fold 1, which explains the poor performance of every model in that fold. This difference may arise because the training data fall largely within the university holidays, while the validation data fall within the teaching period. Fold 1 is therefore excluded when assessing model performance. More generally, because the error in each fold depends strongly on the data in that fold, single-fold errors should be interpreted with caution when judging model performance.

6 Optimisation and Performance

6.1 Depth Sweep

The depth of the decision tree must be controlled for to prevent the model overfitting to the data. In this project the depth was selected by running the model through many different depths and observing the difference in how the validation and training data converge, as seen in Figure 5.



(a) Coefficient of Determination

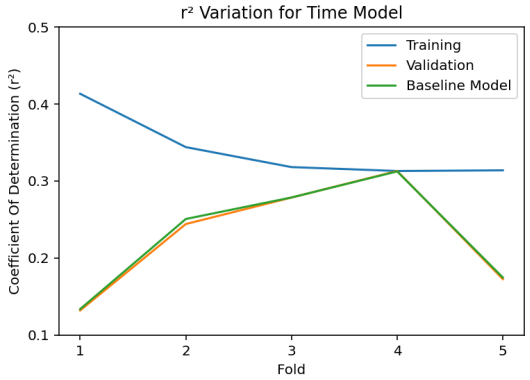
(b) Root Mean Square Error

Figure 5: Model Performance variation with depth

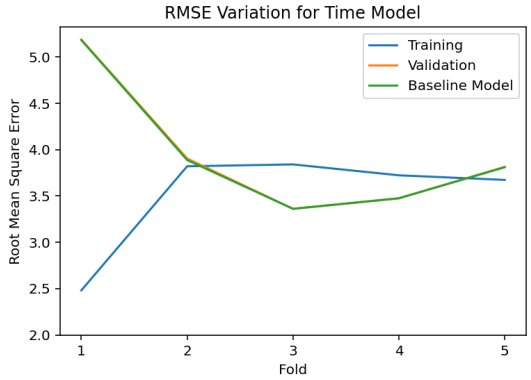
As seen above, the model does rapidly improve until a depth of 3 is reached, and further increasing depth does not dramatically improve model performance. As a result, for the remainder of the project, a model depth of 3 was used.

6.2 Model Complexity Investigation

After determining the baseline model and selecting the modelling approach, the desired complexity of the model was investigated. Three different levels of complexity were investigated by creating a model with inputs of only time or day and time or day, time and calendar context. These models were chosen, as each added level of complexity gives additional information to the model, which should improve accuracy.

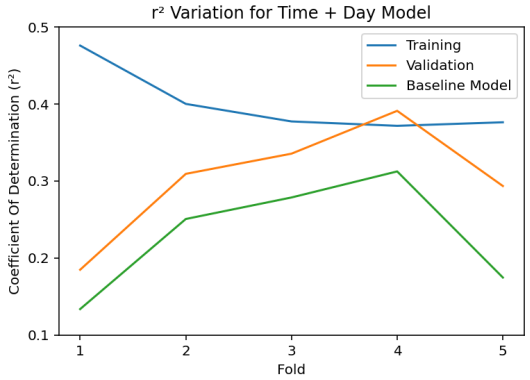


(a) Coefficient of Determination

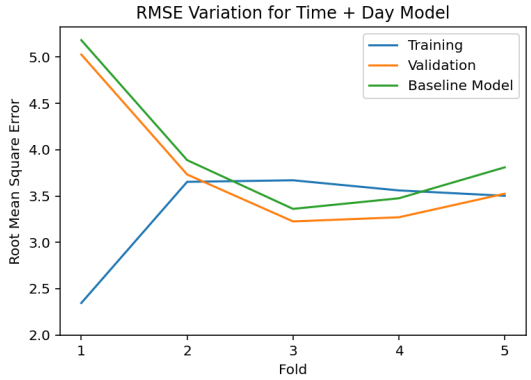


(b) Root Mean Square Error

Figure 6: Performance of Time Only Model

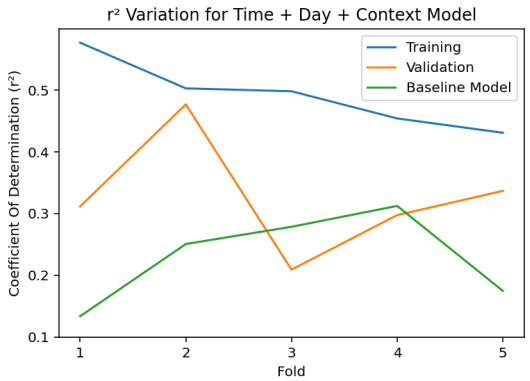


(a) Coefficient of Determination

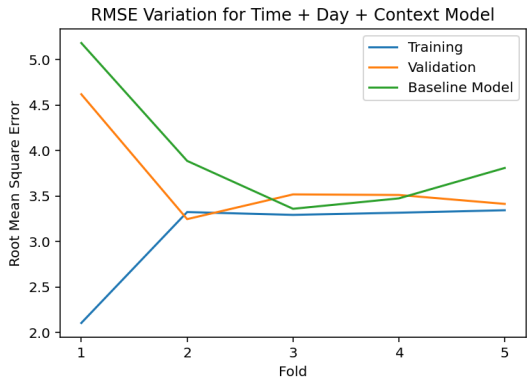


(b) Root Mean Square Error

Figure 7: Performance of Time and Day Model



(a) Coefficient of Determination



(b) Root Mean Square Error

Figure 8: Performance of Time, Day and Context Model

As can be seen in Figure 6, the decision tree model has almost exactly the same performance as the baseline model. The decision tree model uses the average value on each leaf as its prediction, thus by only providing time information, the model approximates the baseline model in its predictions. This project concluded to improve performance, the model must be provided more information about the state of the building.

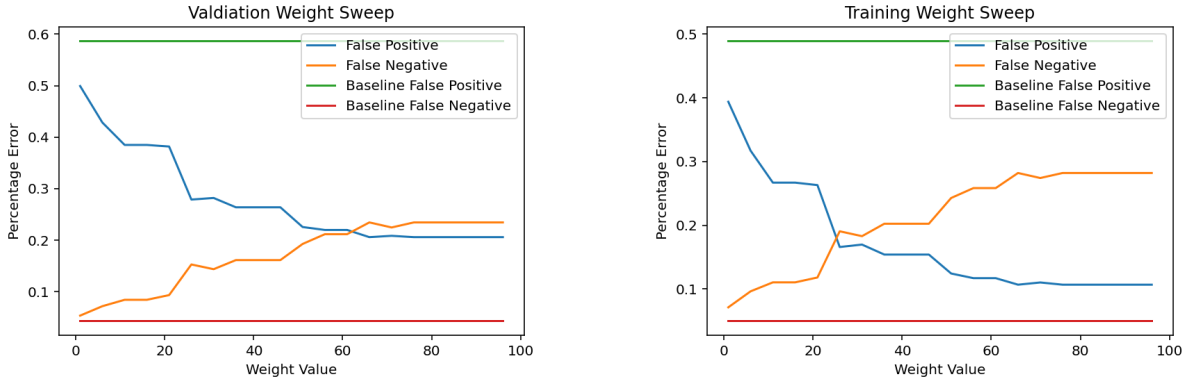
The time and day model used in Figure 7 has added information about the day of the week, which improves its ability to capture the variance in the data compared to the time only model. Additionally, it has a consistent improvement in error compared to the baseline model. Ultimately, this model is a moderate improvement to the baseline model while still failing to meet the secondary goal of a coefficient of determination of 0.5.

The most complex model that was investigated used the day, time and Class and Exam flags that occur when university classes or exams are running at Monash. The dates for the university timetable were collected from Monash's website. However, as can be seen in Figure 8, the model captures an inconsistent amount of the validation variance despite the improvement in capturing the training variance. As the coefficient of determination changes dramatically for every validation fold, it is likely that the model is overfitting to the training data and losing generality. Additionally, the validation error produced by the model is worse than that of the Time and Day model. The reduced performance could be explained by the fact that the building is not primarily used by students. The demographic that uses the building used in this project is not known due to poor database documentation, which means that the assumption that the building is used by students could be flawed. Ultimately, the class and exam flags only introduce more uncorrelated variance into the model, resulting in a decrease in performance. As a result, for future improvements the demographic and schedule of the target building should be considered.

The result of the complexity investigation concluded that a decision tree model with the inputs of the time of day and day of the week is the most appropriate model to use in this application.

6.3 Weight Sweep

The weightings of the decision tree can encourage the model to fit particular values more strongly, which is useful for this project to achieve the primary goal. To increase performance times when the building was unoccupied were assigned an increased weight value, and the change in the performance of the model is summarised in Figure 9.



(a) Validation of Determination

(b) Root Mean Square Error

Figure 9: Model Performance variation with depth

As shown in Figure 9a, the model satisfies the primary goal of correctly predicting 75% of the periods when the building is unoccupied when a weight of 65 is selected, failing to identify only 22% of empty periods. The figure also shows a corresponding increase in false negatives, although these remain at a reasonable level. Additionally, every weight value produces false positives at a lower rate than the baseline model, which demonstrates the main advantage of using a decision tree in this project.

Weight Value	Coefficient of Determination	RMSE Error
1	0.29	3.52
65	0.27	3.57

Table 8: Impact of weight value on accuracy of model

As can be seen in Table 8 the change of the weight parameter does not significantly impact the accuracy of the model. As a result, a weight value of 65 was chosen so that the primary goal was achieved.

7 Final Model Summary

Model Type	Decison Tree Model	
Input	Time of day in hours	Float
Depth	A depth of three was selected	
Input	The day of the week as a number (Monday=0 through to Sunday=6)	Integer
Unoccupied Time Weight	A weight of 65 was given to times where the building is empty	
Output	Number of people in the building	Float

Table 9: Final Model Summary

8 AI use and Reflection

Claude (Anthropic) was used throughout the project to debug code, produce documentation, critique the validation design and proofread report text. Its approximate contribution was 30% of the overall effort, mainly in debugging and wording. AI was used for these tasks because it sped up the diagnosis of errors, such as array shape errors and swapped polyval arguments, and flagged methodological problems that I then confirmed myself. These included that a single chronological split made the validation error fall below the training error, and that zero-filling gaps distorted the baseline.

All AI suggestions were checked by re-running the code, inspecting outputs and plots, and comparing them against the data. Some suggestions were unhelpful or wrong. For example, the AI incorrectly identified the cause of an unexpected result because it misunderstood which model was being used. The problem framing, target goals, modelling choices and reported claims remained my responsibility. Using AI improved my understanding of time-series validation and made me more careful to verify results independently.

Overall, this project taught me that AI is most useful for small problems with a single solution, and less useful for large, complex ones. To get the best results, I broke each large problem into smaller problems that AI could help with. I then combined these solutions to solve the original problem.